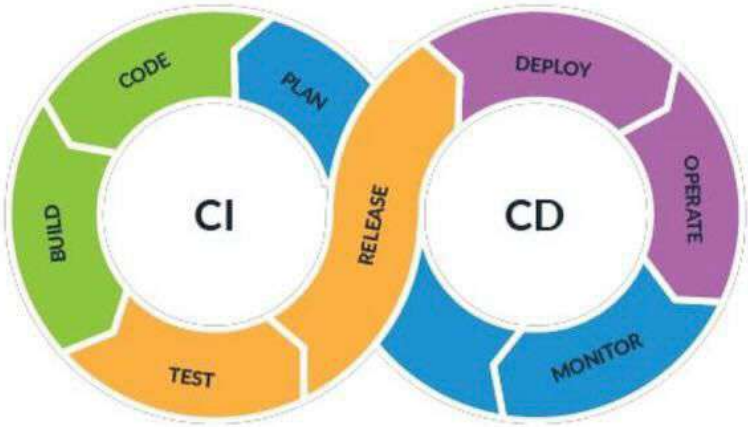
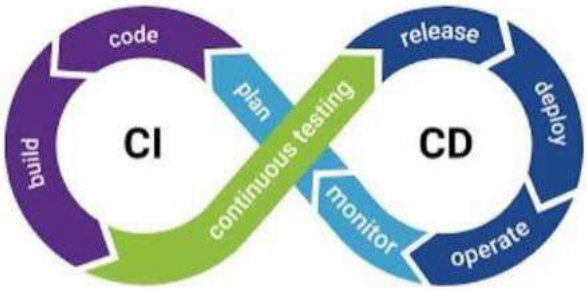
CI CD PIPELINE



1. Introduction

1.1 What is CI/CD?

- CI/CD (Continuous Integration and Continuous Delivery/Deployment) is a DevOps practice that automates the entire software development lifecycle from integrating code, to testing it, to deploying it into production.
- It removes manual steps and ensures code moves from developer → testing → production reliably.



1.2 Why CI/CD Matters

- **Faster and More Frequent Releases**
  Traditional releases were slow and manual. CI/CD automates the build, testing, and deployment process, allowing teams to release features quickly, deploy multiple times a day and respond faster to market needs.

- **Early Detection of Bugs**
  CI/CD ensures that every code change automatically triggers build, test and validation. This means problems are detected immediately, not at the end of the project.

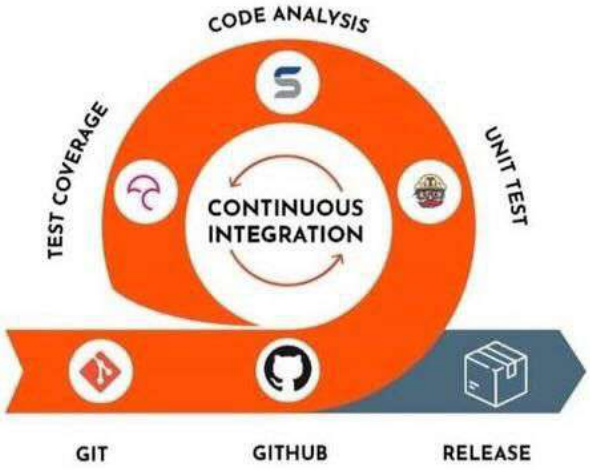
- **Improved Code Quality**
  Code quality is improved by running tests like unit tests, integration tests, UI tests, API tests automatically. Code is always tested before it reaches production.

- **Reduced Manual Work**
  Manual deployments often cause human errors, missed steps, environment misconfiguration. CI/CD automates everything, including build, test, and deployment steps.

2. Core Concepts

2.1 Continuous Integration (CI)

- Continuous Integration is the practice where developers frequently merge their code changes into a shared repository (e.g., GitHub, GitLab, Bitbucket). Code is automatically built and tested every time and issues are detected early.
- CI checks code compilation, dependency installation, unit tests, code quality, formatting checks and Static analysis (SonarQube, ESLint).
- Key benefits of CI are reduces integration conflicts, improves code quality, speeds up development, improves collaboration between developers and QA.

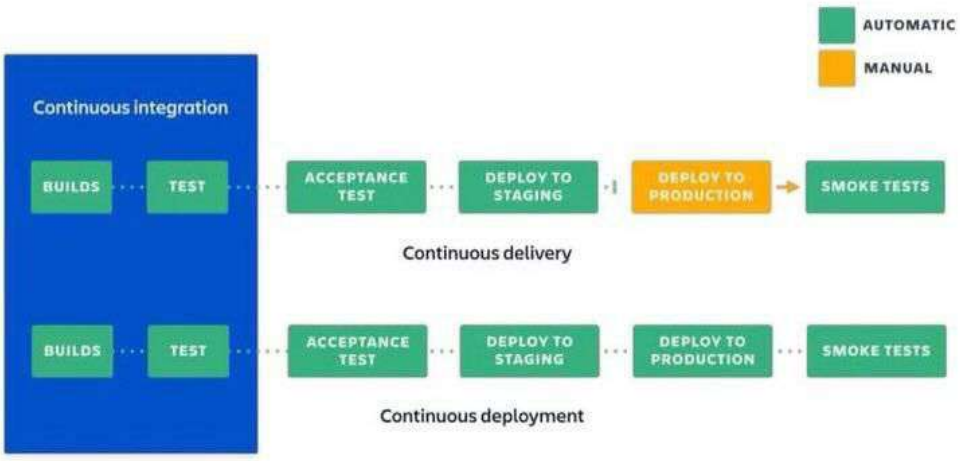


2.2 Continuous Delivery (CD)

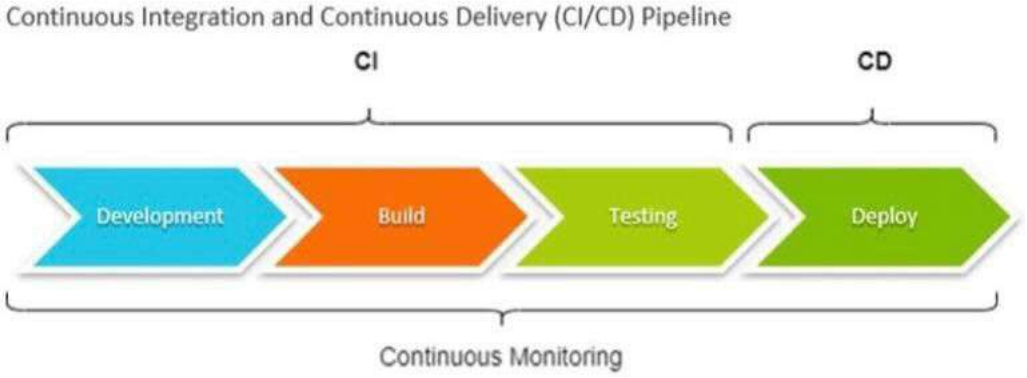
- Continuous Delivery ensures that the application is always in a deployable state.
- After CI successfully validates the code, the application is automatically prepared for release into staging or pre-production environments.
- Deployment of production often requires manual approval, giving teams more control.
- Key benefits are frequent and reliable releases, reduced risks through smaller and incremental updates.

2.3 Continuous Deployment (CD)

- Continuous Deployment is the next step after Continuous Delivery.
- Here, every change that passes automated tests is automatically deployed into production without manual intervention.
- This approach is common in large-scale companies (like Netflix, Amazon, Google) that require rapid and reliable releases.
- Key benefits are fully automated release process, faster innovation and feature delivery, immediate customer feedback.



3. CI CD Pipeline stages



I. Source Stage (Version Control Trigger)

- The pipeline starts when developers commit code to a Version Control System (VCS) such as Git, GitHub, GitLab, or Bitbucket.
- A Pull Request/Merge Request is created.
- Pipeline is triggered automatically.
- QA team ensures that no broken code enters the main branch.

II. Build Stage

- In this stage, the source code is compiled or packaged into an executable format (such as a JAR file, Docker image, or binary).
- Dependency management is handled using build tools like Maven, Gradle, or npm, which ensure all external libraries are included.
- Early quality checks are performed, such as linting, static code analysis (SonarQube), and unit tests.
- If the build fails, testing stops.

III. Test Stage

- Once the building is ready, automated tests validate functionality, integration, and performance.
- Different types of tests executed at this stage include:
  - o **Unit Tests**: Validate individual functions or modules.
  - o **Integration Tests**: Check if different components interact correctly.
  - o **Regression Tests**: Ensure new changes do not break existing functionality.
  - o **UI Tests**: Validate user interface behavior across browsers/devices (e.g., Selenium, Cypress).

- **API Tests**: Validate endpoints and responses (e.g., Postman, RestAssured).
- **Performance Tests**: Check load, stress, and scalability (e.g., JMeter).

- This is the main stage where QA engineers contribute.

IV. Static Code Analysis (Quality Gates)

- Tools like SonarQube check for code quality.
- In here QA team ensures code meets quality standards.
- Pipeline can stop if quality gates fail.

V. Artifact Packaging & Storage Stage

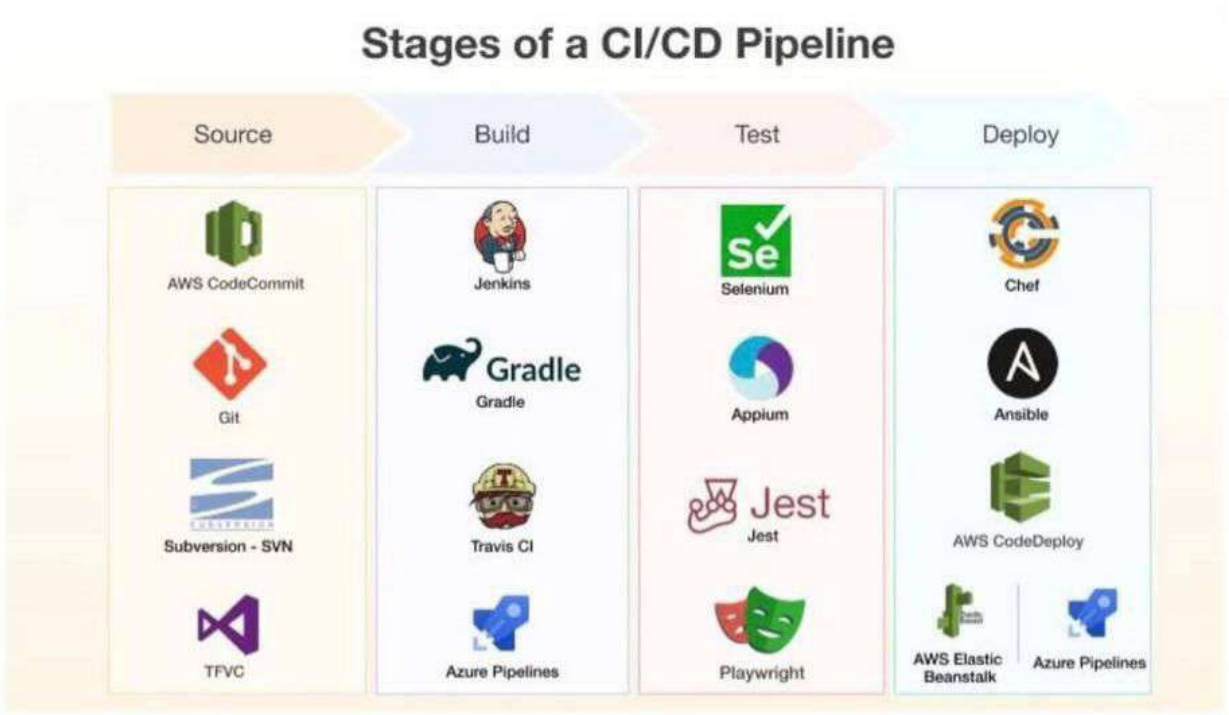
- Once the build + tests pass, the output is stored.
- An artifact is the final packaged output of your application.
  Ex:- .jar, .war, .apk, dist/ folder, Docker image.
- QA ensures that everyone uses the same build.

VI. Deployment Stage

- After passing tests, the application is deployed to staging, pre-production, or production environments.
- QA often runs smoke tests after deployment to ensure the application is working correctly.
- Different deployment strategies are used to minimize downtime and risks:
  - Blue-Green Deployment: Two environments exist (Blue = current, Green = new). Traffic switches to Green once testing is successful.
  - Canary Deployment: New version is rolled out to a small percentage of users before a full rollout.
  - Rolling Deployment: Old versions are gradually replaced with new ones in batches.

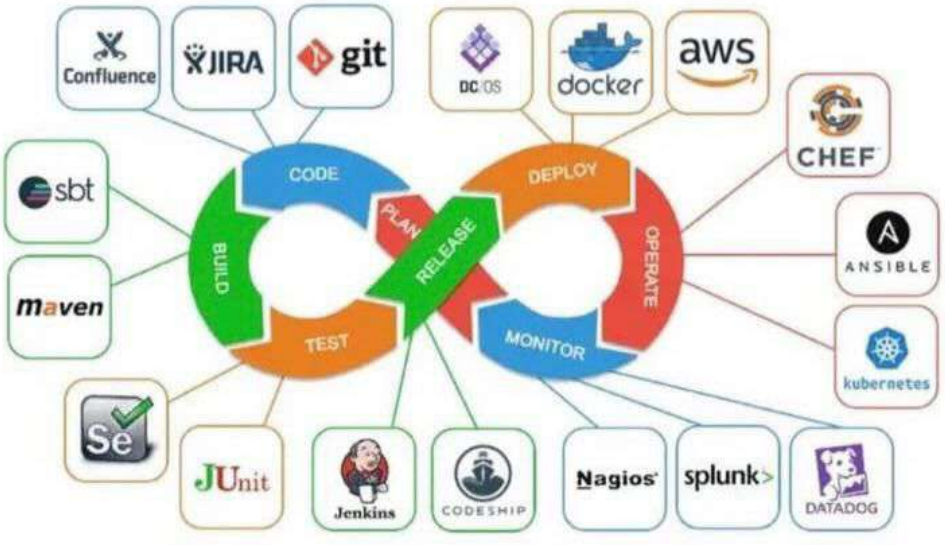
VII. Monitoring & Feedback Stage

- After deployment, systems are monitored.
- Once the application is alive, monitoring is essential to ensure stability and performance.
- Continuous feedback loops help detect issues early in production, allowing teams to roll back or fix bugs quickly.
- QA and Ops teams perform post-deployment validation (sanity checks, exploratory testing) to confirm that the release meets user expectations.



4. CI/CD Tools

CI/CD tools are technologies that help teams automate the process of building, testing, and deploying software. They reduce manual work, speed up delivery, and improve software quality.



4.1 CI/CD Platforms

These are the main tools that run your pipeline (build, test, deploy).

❖ **Jenkins**

- Jenkins is the most widely used CI/CD tool in the world.
- Open-source automation server.
- Highly customizable with 1,800+ plugins.
- Runs pipelines defined as Jenkinsfile.
- Works with any language or framework.
- Can run on your own server or cloud machine.
- Easily integrates with Selenium, Rest Assured, JMeter, Newman.

❖ **GitHub Actions**

- GitHub Actions is a cloud-native CI/CD tool built directly into GitHub.
- Pipelines written in **YAML**.
- Automatically triggers on push or pull request.
- Integrates easily with GitHub repos.
- Reusable workflow templates.

❖ **Azure DevOps Pipelines**

- Enterprise-level CI/CD system by Microsoft.
- Multi-stage pipelines.
- Works with Azure cloud services.
- Excellent for large enterprise QA teams.
- Built-in test management.

❖ **Bitbucket Pipelines**

- A cloud-based CI/CD tool integrated with Bitbucket repositories.
- YAML-based pipelines.
- Easy integration with Jira.
- Runs on Docker containers.

4.2 Build & Dependency Tools

These tools prepare your application for CI by compiling and packaging it.

❖ **Maven**

- Maven is a build automation tool for Java.
- Uses pom.xml to manage dependencies.
- Automatically downloads libraries.
- Builds .jar and .war files.

❖ **Gradle**

- A modern build tool used for Java, Kotlin, and Android.
- Faster than Maven.
- Uses build.gradle.
- QA Engineers use heavily in android automation.

❖ **npm/ yarn**

- Tools to build JavaScript applications.
- npm = Node Package Manager
- yarn = Faster alternative
- Used for React, Angular, Vue builds

❖ **Docker Hub / AWS ECR / GCR**

- Repositories for storing Docker images.
- QA can deploy the exact same container version used in production.

4.3 Artifact Repository Tools

These tools store the final build (called artifacts).

❖ **JFrog Artifactory**

- A universal artifact repository.
- Stores .jar, .war, .apk, Docker images, anything.
- Integrates with Jenkins, GitHub, GitLab.
- QA always tests from stored artifacts, not developer builds.

❖ **Docker Hub / AWS ECR / GCR**

- Repositories for storing Docker images.
- QA can deploy the exact same container version used in production.

❖ **Nexus Repository Manager**

- Another widely used artifact repository.
- Works with Maven, npm, Docker images.
- Easy to configure with CI tools.
- Helps in regression and rollback testing.

4.4 Testing Tools Used in CI/CD Pipelines

These are heavily used by QA for automation inside pipelines.

❖ **Selenium**

- UI automation for web applications.
- Automates browser interactions.
- Used in CI for running UI regressions.

❖ **Playwright**

- Modern framework for reliable UI tests.
- Runs parallel tests fast.
- Good for CI pipelines.

❖ **Postman + Newman**

- API testing tool and CLI runner.
- Run Postman collections inside pipelines.
- Automatically generate API test reports.

❖ **JMeter**

- Performance/load testing tool.
- Runs load tests in CI to check performance.

4.5 Code Quality & Security Tools (Quality Gates)

These tools enforce quality before merging code.

❖ **SonarQube**

- Checks code quality, bugs and vulnerabilities.
- Measures test coverage.
- Blocks bad code from merging
- QA ensures quality gate rules are followed.

❖ **Snyk**

- Security Scanning Tool.
- QA detects vulnerabilities early in CI pipeline.

4.6 Deployment Tools (CD Phase)

These tools deliver the application to staging/production.

❖ **Docker**

- Packages app into portable containers.
- Works on any system the same way.
- No "works on my machine" issues.

❖ **Kubernetes (K8s)**

- Used for large-scale deployments.
- QA can test apps running on Kubernetes clusters.

❖ **AWS CodeDeploy / Azure App Service**

- Cloud deployment tools.
- QA validates different environments (DEV, QA, PROD) deployed automatically.

5. Deployment Pipelines

A deployment pipeline automates how a new software version moves from CI → testing → staging → production.

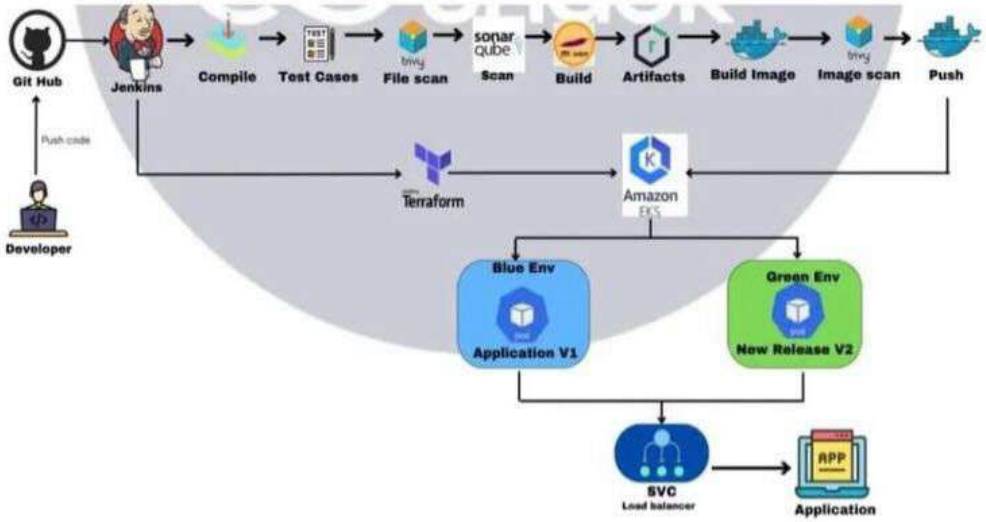
5.1 Blue/Green deployments

A strategy where two production environments exist:

- **Blue** = current live environment
- **Green** = new version of the application
- QA test the new version in green.
- Blue remains as a backup.

Key Benefits

- Users experience no interruptions during updates.
- Immediate rollback is possible if problems occur.
- Deployments can be done without rush, with full control over the process.

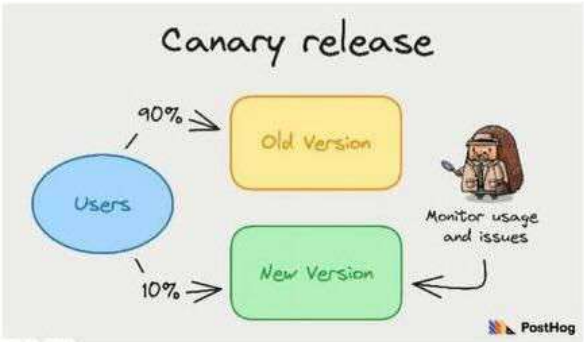


5.2 Canary Releases

- Deploying the new version to a small percentage of users first.
- Monitoring error rates, performance, logs.
- Slowly expanding rollout.
- If any issue detected stopping the rollout.

Key Benefits

- Detect problems early.
- Gives real world testing before full release.
- Lower risk of failures.



5.3 Rollbacks

Reverting to the previous stable version when a deployment fails.

- **Manual rollback** – Developer/DevOps reverts version
- **Automatic rollback** – System detects failure and auto-reverts
- **Switch-back rollback** – In Blue/Green deployments, simply flip from Green → Blue

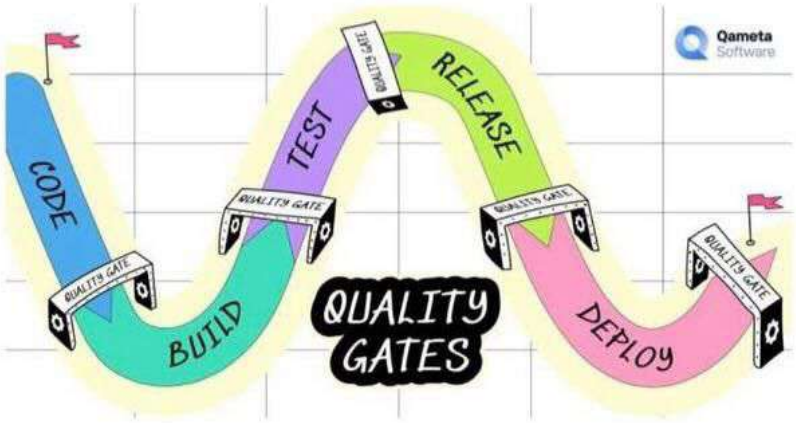
Rollback triggers high error rate, failing health checks, failed automated tests and performance degradation.

5.4 Approvals in Deployment Stages (QA Approval Gates)

A mandatory checkpoint where a QA must approve before deployment continues. This is also called a manual gate or approval gate.

How it works,

- CI builds and tests the application
- CD pipeline pauses at the approval gate
- QA reviews test results, logs, metrics, build quality
- QA clicks Approve or Reject



6. Roles Responsible for a CI/CD Pipeline

6.1 Developers

- Write and commit code to the repository.
- Create and maintain unit tests.
- Collaborate with QA to make sure test cases are automated.

6.2 Quality Assurance (QA) Engineers

- Design and maintain test cases (manual + automated).
- Integrate automated tests (API, UI, regression, performance) into the pipeline.
- Define quality gates (e.g., build fails if less than 80% test coverage).
- Perform exploratory testing and validate staging environments.
- Provide continuous feedback on bugs and defects.

6.3 DevOps Engineers

- Set up and maintain the pipeline infrastructure (Jenkins, GitLab CI, GitHub Actions, etc.).
- Manage build servers, deployment scripts, and environments.
- Ensure continuous integration of tools, monitoring, and security checks run smoothly.
- Work with developers and QA to automate deployments and scaling.

6.4 Operations / SRE (Site Reliability Engineers)

- Monitor application in production after deployment.
- Ensure availability, performance, and logging.
- Collaborate with QA to analyze post-release issues.
- Define rollback strategies in case of failures.

6.5 Project Managers / Product Owners

- Define release schedules and sprint goals.
- Coordinate between Dev, QA, and DevOps teams.
- Approve production releases in Continuous Delivery setups (where manual approval is required).